Principles of Database Systems (CS307)

Lecture 4: Intermediate and Advanced SQL

Zhong-Qiu Wang

Department of Computer Science and Engineering
Southern University of Science and Technology

- Most contents are from slides made by Stéphane Faroult and the authors of Database System Concepts (7th Edition).
- Their original slides have been modified to adapt to the schedule of CS307 at SUSTech.
- The slides are largely based on the slides provided by Dr. Yuxin Ma

Announcements

- First assignment will come out later today, due date: TBD
  - Do not miss the deadline, or you will receive reduced scores

Window Function

Scalar Functions and Aggregation Functions

- Scalar function
  - Functions that operate on values in the current row

```
round(3.141592, 3)  -- 3.142
trunc(3.141592, 3)  -- 3.141
```

```
upper('Citizen Kane')
lower('Citizen Kane')
substr('Citizen Kane', 5, 3)  -- 'zen'
trim('   Oops  ')  -- 'Oops'
replace('Sheep', 'ee', 'i')  -- 'Ship'
```

- Aggregation function
  - Functions that operate on sets of rows and return an aggregated value

```
count(*)/count(col), min(col), max(col), stddev(col), avg(col)
```

Issues with Aggregate Functions

- A Problem: In aggregated functions, the details of the rows are vanished
  - For example: if we ask for the year of the oldest movie per country
    - We get a country, a year, and nothing else.

```
select country,
       min(year_released) earliest_year
from movies
group by country
```

country	oldest_movie
fr	1896
ke	2008
si	2000
eg	1949
nz	1981
bg	1967
ru	1924
gh	2012
pe	2004
hr	1970
sg	2002
mx	1933
cn	1913
ee	2007
sp	1933
cl	1926
ec	1999
cz	1949
dk	1910
vn	1992
ro	1964
mn	2007
gb	1916
se	1913
tw	1971
ie	1970
ph	1975
ar	1945
th	1971

Issues with Aggregate Functions

- A Problem: In aggregated functions, the details of the rows are vanished
  - For example: if we ask for the year of the oldest movie per country
    - We get a country, a year, and nothing else

If we want some more details, like the title of the oldest movies for each country

- We can only use self-join to get the title information
- The join conditions include (1) same country code; and (2) same release year

```
select m1.country,
       m1.title,
       m1.year_released
from movies m1
     inner join
     (select country,
        min(year_released) minyear
        from movies
        group by country) m2
on m2.country = m1.country and m2.minyear = m1.year_released
order by m1.country
```

Issues with Aggregate Functions

- A Problem: In aggregated functions, the details of the rows are vanished
  - For example: if we ask for the year of the oldest movie per country
    - We get a country, a year, and nothing else

If we want some more details, like the title of the oldest movies for each country

- What if the title of the second oldest movie is what we want?

```
select m1.country,
       m1.title,
       m1.year_released
from movies m1
     inner join
     (select country,
        min(year_released) minyear
        from movies
        group by country) m2
on m2.country = m1.country and m2.minyear = m1.year_released
order by m1.country
```

Issues with Aggregate Functions

- A Problem
  - In aggregated functions, the details of the rows are vanished
  - Another example
    - How can we rank the movies in each country separately based on the released year?
    - “order by” for subgroups
  - One more example
    - Get the top-3 oldest movies for each country
    - How can we implement it?

Window Function

- Syntax:

```
<function> over (partition by <col_p> order by <col_o1, col_o2, …>)
```

  - `<function>`
    - Ranking window functions
    - Aggregation functions
  - `partition by`
    - Specify the column for grouping
  - `order by`
    - Specify the column(s) for ordering in each group

Ranking Window Function

- Example
  - How can we rank the movies in each country separately based on the released year?
    - “order by” for subgroups

```
select country,
       title,
       year_released,
       rank() over (
           partition by country order by year_released
       ) oldest_movie_per_country
from movies;
```

Partitioned by country
- i.e., a country in a group

country	title	year_released	oldest_movie_per_country
ar	some title	1948	1
ar	some title	1959	2
ar	some title	1980	3
cn	some title	1987	1
cn	some title	2002	2
uk	some title	1985	1
uk	some title	1992	2
uk	some title	2010	3

`rank()`
- A function to say that “I want to order the rows in each partition”
- No parameters in the parentheses

`order by year_released`
- In each group (partition), the rows will be ordered by the column “`year_released`”

An order value is computed for each row in a partition.
- Only inside the partition, not across the entire result set

Ranking Window Function

- Example
  - How can we rank the movies in each country separately based on the released year?
    - “order by” for subgroups

```
select country,
       title,
       year_released,
       rank() over (
           partition by country order by year_released
       ) oldest_movie_per_country
from movies;
```

country	title	year_released	oldest_movie_per_country
ar	some title	1948	1
ar	some title	1959	2
ar	some title	1980	3
cn	some title	1987	1
cn	some title	2002	2
uk	some title	1985	1
uk	some title	1992	2
uk	some title	2010	3

Note: partition functions can only be used in the `select` clause

- … since it is designed to work on the query result

Ranking Window Function

- Example
  - How can we rank the movies in each country separately based on the released year?
    - “order by” for subgroups

```
select country,
       title,
       year_released,
       rank() over (
           partition by country order by year_released
       ) oldest_movie_per_country
from movies;
```

You can also add “desc” here, similar to the “order by” we introduced before

country	title	year_released	oldest_movie_per_country
ar	some title	1948	1
ar	some title	1959	2
ar	some title	1980	3
cn	some title	1987	1
cn	some title	2002	2
uk	some title	1985	1
uk	some title	1992	2
uk	some title	2010	3

Ranking Window Function

- Why window function, not group by?
  - “Group by” reduces the rows in a group (partition) into one result, which is the meaning of “aggregation”
    - Then, the values in non-aggregating columns are vanished
  - Window functions do not reduce the rows
    - Instead, they attach computed values next to the rows in a group (partition) and keep the details
    - Actually, the partition here means “window”: an affective range for statistics

Ranking Window Function

- Some more ranking window functions
  - Besides `rank()`, we also have `dense_rank()` and `row_number()`
  - The difference is about how they treat rows with the same rank

```
select country,
       title,
       year_released,

       rank() over (
           partition by country order by year_released
       ) rank_result,

       dense_rank() over (
           partition by country order by year_released
       ) dense_rank_result,

       row_number() over (
           partition by country order by year_released
       ) row_number_result
from movies;
```

country	title	year_released	rank_result	dense_rank_result	row_number_result
cn	some title	1948	1	1	1
cn	some title	1959	2	2	2
cn	some title	1959	2	2	3
cn	some title	1987	4	3	4
cn	some title	2002	5	4	5
uk	some title	1985	1	1	1
uk	some title	1992	2	2	2
uk	some title	2010	3	3	3

Aggregation Functions as Window Functions

- Aggregation functions can be used with window functions
  - `min/max(col)` , `sum(col)`, `count(col)`, `avg(col)`, `stddev(col)`, etc.
  - **If `order by` is specified,** return aggregation value from the first row to current row in each partition
  - **If not**, fill all rows with a single aggregation value computed on all rows in each partition

```
select country,
       title,
       year_released,
       sum(runtime) over (
           partition by country order by year_released
       ) total_runtime_till_this_row
from movies;
```

Need to specify a column in the parameter list

	country	title	year_released	total_runtime_till_this_row
1	am	Sayat Nova	1969	78
2	ar	Pampa bárbara	1945	98
3	ar	Albéniz	1947	308
4	ar	Madame Bovary	1947	308
5	ar	La bestia debe morir	1952	494
6	ar	Las aguas bajan turbias	1952	494
7	ar	Intermezzo criminal	1953	494
8	ar	La casa del ángel	1957	570
9	ar	Bajo un mismo rostro	1962	695
10	ar	Las aventuras del Capitán Piluso	1963	785
11	ar	Savage Pampas	1966	897
12	ar	La hora de los hornos	1968	1157
13	ar	Waiting for the Hearse	1985	1354
14	ar	La historia oficial	1985	1354

Pay attention to the behavior on rows with the same rank:

- They are “treated like the same row” here

Aggregation Functions as Window Functions

- Aggregation functions can be used with window functions
  - `min/max(col)` , `sum(col)`, `count(col)`, `avg(col)`, `stddev(col)`, etc.
  - **If `order by` is specified,** return aggregation value from the first row to current row in each partition
  - **If not**, fill all rows with a single aggregation value computed on all rows in each partition

```
select country,
       title,
       year_released,
       min(year_released) over (
           partition by country order by year_released
       ) oldest_movie_per_country
from movies;
```

	cou...	title	year_released	oldest_movie_per_country
1	am	Sayat Nova	1969	1969
2	ar	Pampa bárbara	1945	1945
3	ar	Albéniz	1947	1945
4	ar	Madame Bovary	1947	1945
5	ar	La bestia debe morir	1952	1945
6	ar	Las aguas bajan turbias	1952	1945
7	ar	Intermezzo criminal	1953	1945
8	ar	La casa del ángel	1957	1945
9	ar	Bajo un mismo rostro	1962	1945
10	ar	Las aventuras del Capitán Piluso	1963	1945
11	ar	Savage Pampas	1966	1945
12	ar	La hora de los hornos	1968	1945
13	ar	Waiting for the Hearse	1985	1945
14	ar	La historia oficial	1985	1945

Exercise

- Question: How can we get the top-5 most recent movies for each country?
  - Hint: Use a subquery in the "`from`" clause

```sql
select x.country,
       x.title,
       x.year_released
from (
 select country,
        title,
        year_released,
        row_number()
        over (partition by country
              order by year_released desc) rn
 from movies) x
where x.rn <= 5
```

Update and Delete

So Far...

- We have learned:
  - How to access existing data in tables (select)
  - How to create new rows (insert)

- CRUD/CURD
  - create, read, update, delete
    - In SQL: insert, select, update, delete
    - In RESTful API: Post, Get, Put, Delete
  - Necessary operations for persistent storage

Update

- Make changes to the existing rows in a table
- `update` is the command that changes column values
  - You can even set a non-mandatory column to NULL
  - The change is applied to all rows selected by the `where`

```
update table_name
set column_name = new_value,
    other_col = other_new_val,
    ...
where ...
```

Update

- Remember
  - When you are doing any experiments with writing operations (update, delete), **backup the data first**
    - E.g., copy the tables

Update

- Example: In many Western cultures, a nobiliary particle is used in a surname or family name to signal the nobility of a family
  - We may want to modify some names in such a way as they sort as they should
    - **`von Neumann -> Neumann (von)`**

23 rows | Tx: Auto | DDL

	peopleid	first_name	surname	born	died	gender
1	16439	Axel	von Ambesser	1910	1988	M
2	16440	Daniel	von Bargen	1950	2015	M
3	16441	Eduard	von Borsody	1898	1970	M
4	16442	Suzanne	von Borsody	1957	<null>	F
5	16443	Tomas	von Brömssen	1943	<null>	M
6	16444	Erik	von Detten	1982	<null>	M
7	16445	Theodore	von Eltz	1893	1964	M
8	16446	Gunther	von Fritsch	1906	1988	M
9	16447	Katja	von Garnier	1966	<null>	F
10	16448	Harry	von Meter	1871	1956	M
11	16449	Jenna	von Oÿ	1977	<null>	F
12	16450	Alicia	von Rittberg	1993	<null>	F
13	16451	Daisy	von Scherler Mayer	1966	<null>	F
14	16452	Gustav	von Seyffertitz	1862	1943	M
15	16453	Josef	von Sternberg	1894	1969	M

John von Neumann

Update

- Example: In many Western cultures, a nobiliary particle is used in a surname or family name to signal the nobility of a family
  - We may want to modify some names in such a way as they sort as they should
    - `von Neumann -> Neumann (von)`

- First, how can we find these names?

Update

- Example: In many Western cultures, a nobiliary particle is used in a surname or family name to signal the nobility of a family
  - We may want to modify some names in such a way as they sort as they should
    - **`von Neumann -> Neumann (von)`**
- First, how can we find these names?
  - Wildcards
  - Strings starting with “von ”

```
select * from people_1 where surname like 'von %';
```

Update

- Example: In many Western cultures, a nobiliary particle is used in a surname or family name to signal the nobility of a family
  - We may want to modify some names in such a way as they sort as they should.
    - `von Neumann -> Neumann (von)`
- Then, how should we update the names?

```
(first_name) John                    (first_name) John
(surname) von Neumann      ⇨         (surname) Neumann (von)
```

  - Try the transformation with select:

```
select replace('von Neumann', 'von ', '') || ' (von)';
```

	?column?
1	Neumann (von)

Update

- Example: In many Western cultures, a nobiliary particle is used in a surname or family name to signal the nobility of a family
  - We may want to modify some names in such a way as they sort as they should.
    - **`von Neumann -> Neumann (von)`**
- Finally, the `update` statement:

```
-- Specify the table
update people

-- Set the update rule
set surname = replace(surname, 'von ', '') || ' (von)'

-- Find the rows that need to be updated
where surname like 'von %';
```

Update

- The `where` clause specifies the affected rows
  - If `where` is not provided, the update will be applied to all rows in the table
    - Use with caution!
    - Sometimes, there will be a warning in IDEs such as DataGrip

Update

- The update operation may not be successful when constraints are violated
  - E.g., update the primary key but with duplicated values

```
update people set peopleid = 1 where peopleid < 10;
```

```
[23505] ERROR: duplicate key value violates unique constraint "people_pkey"
Detail: Key (peopleid)=(1) already exists.
```

  - This is why we need constraints when creating tables
    - Avoid unacceptable writing operations that break the integrity of the tables

Update

- Use Subqueries in update
  - Complex update operations where values are based on a query result
- Example: Add a column in people table to record the number of movies one has joined (either directed or played a role in)

```
update table_name
set column_name = new_value,
    other_col = other_new_val,
    ...
where ...
```

Update

- Example: Add a column in people table to record the number of movies one has joined (either directed or played a role in)
  - First, count the movies for a person
    - Used as the subquery part in the update statement

```
select count(*) from credits c where c.peopleid = [some peopleid];
```

Update

- Example: Add a column in people table to record the number of movies one has joined (either directed or played a role in)
  - First, count the movies for a person
    - Used as the subquery part in the update statement
  - Then, update

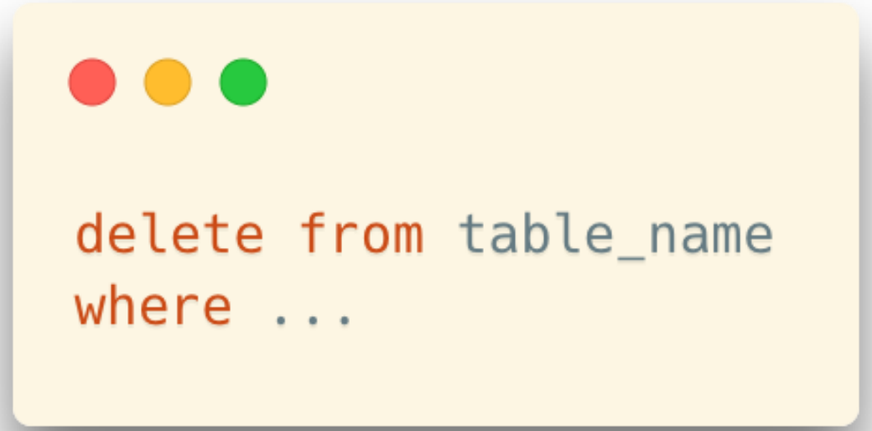
```
update people p

set num_movies = (
    select count(*) from credits c where c.peopleid = p.peopleid
)

where peopleid < 500;
-- This where is only for testing purpose;
-- You should change it (or remove it) when in actual use.
```

Delete

- As the name shows, `delete` removes rows from tables



  - If you omit the WHERE clause, then (as with UPDATE) the statement affects all rows and you **end up with an empty table!**
  - Well,
    - Many database products provide a roll-back mechanism when deleting rows
    - Transactions can also protect you (to some extent)

Constraints

- One important point with constraints (esp., foreign keys) is that they guarantee that data remains consistent
  - They not only work with `insert`, but with `update` and `delete`
  - Example: Try to delete some rows in the `country` table
    - Foreign-key constraints are especially useful in controlling `delete` operations

```
delete from countries where country_code = 'us';
```

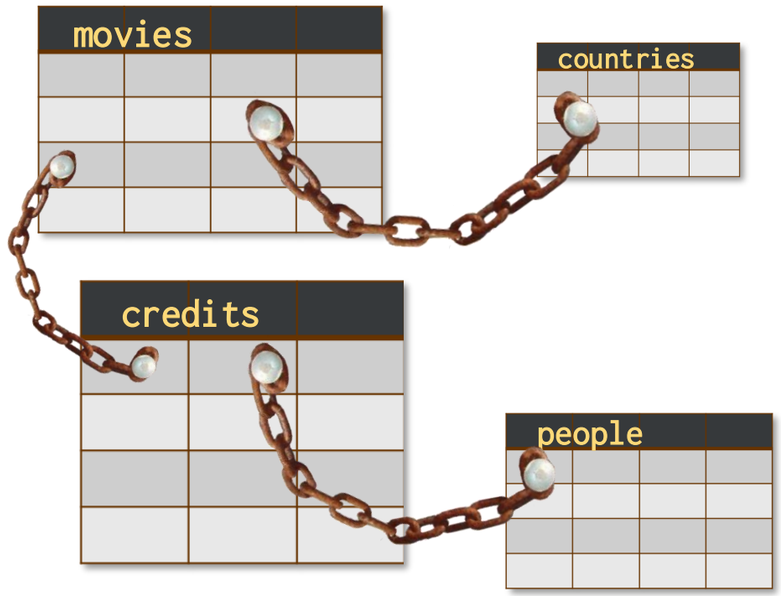
```
[23503] ERROR: update or delete on table "countries" violates foreign key constraint "movies_country_fkey" on table "movies"
Detail: Key (country_code)=(us) is still referenced from table "movies".
```

movieid	title	country	year_released
1	Casab	us	1942
2	Goodfellas	us	1990
3	Bronenosets Potyomkin	ru	1925

country_code	country_name	continent
ru	Russia	Europe
us	United States	America

Constraints

- This is why constraints are so important:
  - They ensure that whatever happens, you'll always be able to make sense of ALL pieces of data in your database
  - For any changes made to the tables, all declared constraints need to be satisfied



Function

Built-in Functions

- Most DBMS provides a series of built-in functions
  - E.g., Scalar function, aggregation function, window function

```
round(3.141592, 3)  -- 3.142
trunc(3.141592, 3)  -- 3.141
```

```
upper('Citizen Kane')
lower('Citizen Kane')
substr('Citizen Kane', 5, 3)  -- 'zen'
trim('   Oops  ')  -- 'Oops'
replace('Sheep', 'ee', 'i')  -- 'Ship'
```

```
count(*)/count(col), min(col), max(col), stddev(col), avg(col)
```

```
<function> over (partition by <col_p> order by <col_o1, col_o2, …>)
```

- <function>: we can apply (1) ranking window functions, or (2) aggregation functions
- partition by: specify the column for grouping
- order by: specify the column(s) for ordering in each group

Self-defined Function

- Sometimes the built-in functions cannot fulfill our requirements
  - And the power of declarative language (SQL) is not strong enough

- Most DBMS implement a built-in, SQL-based programming language
  - A procedural extension to SQL

Procedural vs. Declarative

- Two different programming paradigms
  - Imperative programming （命令式编程）
    - Describe the algorithms step-by-step (i.e., how to do)
    - Procedural （过程式）: C (and many other legacy languages)
    - Object-oriented: Java
  - Declarative programming（声明式编程）
    - Describe the result without specifying the detailed steps (i.e., what to do)
    - (Pure) declarative: SQL, Regular Expressions, Markup (HTML, XML), CSS
    - Functional: Scheme, Haskell, Scala, Erlang
    - Logic programming: Prolog

Procedural vs. Declarative

- E.g., How can we get a cup of tea?
  - In a procedural way:

1. Get a cup
2. Get some tea
3. Get some hot water
4. Put tea into the cup
5. Pour hot water into the cup
6. `return tea;`

  - In a declarative way:

`<a cup of tea/>`

    - You don't really need to know how to make a cup of tea
    - The system can do it in a black-box manner

大佬喝茶

Procedural vs. Declarative

- E.g., Find all Chinese movies before 1990 in the `movies` table?
  - In a procedural way:

1. Read the `movies` table into the memory
2. For each row `i` in the table, repeat:
2.1 In row `i`, read the value of the column “country”
2.2 if …

  - In a declarative way:

```
select * from movies where country = ‘cn’ and year_released < 1990
```

    - You don’t really need to know how to filter the table
    - The DBMS system can do it in a black-box manner

Procedural vs. Declarative

- Benefits in declarative languages
  - No need to understand the details
    - The systems take in charge of all the details
  - Easier to use than imperative programming
    - More user-friendly
- Problems in declarative languages
  - Cannot specify the control flow of a program
    - If there is no such command as <a cup of tea/>, you need to create it by yourself

Procedural Extension to SQL

- Many DBMS products provide a proprietary procedural extension to the standard SQL
  - Transact-SQL (T-SQL) 

  - PL/SQL 

  - PL/PGSQL 

  - (No specific name) 

  - (Not supported) 

Function in (Postgre)SQL

- Example: Display the full name for people with “von”
  - When introducing `update`, we have modified the names starting with “von” into “… (von)” for ordering
    - **`von Neumann -> Neumann (von)`**

	peopleid	first_name	surname	born	died	gender
1	16439	Axel	Ambesser (von)	1910	1988	M
2	16440	Daniel	Bargen (von)	1950	2015	M
3	16441	Eduard	Borsody (von)	1898	1970	M
4	16442	Suzanne	Borsody (von)	1957	<null>	F
5	16443	Tomas	Brömssen (von)	1943	<null>	M
6	16444	Erik	Detten (von)	1982	<null>	M
7	16445	Theodore	Eltz (von)	1893	1964	M
8	16446	Gunther	Fritsch (von)	1906	1988	M
9	16447	Katja	Garnier (von)	1966	<null>	F
10	16448	Harry	Meter (von)	1871	1956	M
11	16449	Jenna	Oÿ (von)	1977	<null>	F
12	16450	Alicia	Rittberg (von)	1993	<null>	F
13	16451	Daisy	Scherler Mayer (von)	1966	<null>	F
14	16452	Gustav	Seyffertitz (von)	1862	1943	M

Function in (Postgre)SQL

- If we simply concatenate the first name and the last name, it looks like this:
  - A little bit weird format (a trailing “von”)

```
select first_name || ' ' || surname
from people
where surname like '%(von)';
```

	?column?
1	Axel Ambesser (von)
2	Daniel Bargen (von)
3	Eduard Borsody (von)
4	Suzanne Borsody (von)
5	Tomas Brömssen (von)
6	Erik Detten (von)
7	Theodore Eltz (von)
8	Gunther Fritsch (von)
9	Katja Garnier (von)
10	Harry Meter (von)
11	Jenna Oÿ (von)
12	Alicia Rittberg (von)
13	Daisy Scherler Mayer (von)
14	Gustav Seyffertitz (von)

Function in (Postgre)SQL

- Question: How can we restore the format into “**first_name von surname**”?
  - String operations
  - `i.e., Neumann (von) -> von Neumann`

Function in (Postgre)SQL

- Question: How can we restore the format into “**first_name von surname**”?
  - String operations
  - `i.e., Neumann (von) -> von Neumann`

```
select case
    when first_name is null then ''
    else first_name || ' '
end || case position('(' in surname)
    when 0 then surname
    else trim(')' from substr(surname, position('(' in surname) + 1))
        || ' '
        || trim(substr(surname, 1, position('(' in surname) - 1))
end
from people
where surname     like '%(von)';
```

Function in (Postgre)SQL

- Question: How can we restore the format into “**first_name von surname**”?
  - String operations

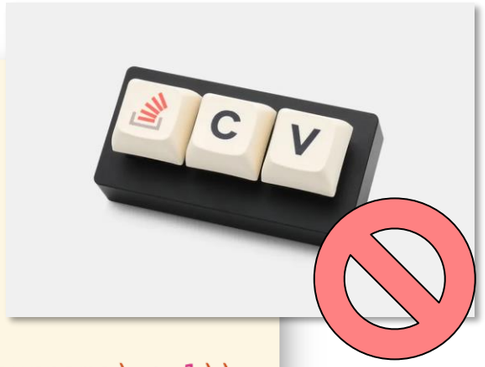
Then, how can we store this part to reuse it in the future?

```
       case
    when first_name is null then ''
    else first_name || ' '
end || case position('(' in surname)
    when 0 then surname
    else trim(')' from substr(surname, position('(' in surname) + 1))
         || ' '
         || trim(substr(surname, 1, position('(' in surname) - 1))
end
from people
where surname     like '%(von)';
```

Function in (Postgre)SQL

- Copy and paste is not a good habit
  - Whenever you have painfully written something as complicated, which is pretty generic, you'd rather not copy and paste the code every time you need it

```
case
    when first_name is null then ''
    else first_name || ' '
end || case position('(' in surname)
    when 0 then surname
    else trim(')' from substr(surname, position('(' in surname) + 1))
        || ' '
        || trim(substr(surname, 1, position('(' in surname) - 1))
end
```



Function in (Postgre)SQL

- Stored for reuse
  - In PostgreSQL, we can store the expression and reuse it in another context
- Self-defined Function
  - `create function`

```
CREATE [OR REPLACE] FUNCTION function_name (arguments)
RETURNS return_datatype AS $variable_name$
   DECLARE
      declaration;
      [...]
   BEGIN
      < function_body >
      [...]
      RETURN { variable_name | value }
   END; LANGUAGE plpgsql;
```

```
CREATE [ OR REPLACE ] FUNCTION
    name ( [ [ argmode ] [ argname ] argtype [ { DEFAULT | = } default_expr ] [, ...] ] )
    [ RETURNS rettype
      | RETURNS TABLE ( column_name column_type [, ...] ) ]
  { LANGUAGE lang_name
    | TRANSFORM { FOR TYPE type_name } [, ... ]
    | WINDOW
    | { IMMUTABLE | STABLE | VOLATILE }
    | [ NOT ] LEAKPROOF
    | { CALLED ON NULL INPUT | RETURNS NULL ON NULL INPUT | STRICT }
    | { [ EXTERNAL ] SECURITY INVOKER | [ EXTERNAL ] SECURITY DEFINER }
    | PARALLEL { UNSAFE | RESTRICTED | SAFE }
    | COST execution_cost
    | ROWS result_rows
    | SUPPORT support_function
    | SET configuration_parameter { TO value | = value | FROM CURRENT }
    | AS 'definition'
    | AS 'obj_file', 'link_symbol'
    | sql_body
  } ...
```

https://www.postgresql.org/docs/current/sql-createfunction.html

...or, a simpler version

Function in (Postgre)SQL

- How do we rewrite the name conversion expression into a function?

```
create function full_name(p_fname varchar, p_sname varchar)
returns varchar
as $$
begin
    return case
        when p_fname is null then ''
        else p_fname || ' '
    end || case position('(' in p_sname)
        when 0 then p_sname
        else trim(')' from substr(p_sname, position('(' in p_sname) + 1))
        || ' '
        || trim(substr(p_sname, 1, position('(' in p_sname) - 1))
    end;
end;
$$ language plpgsql;
```

Function in (Postgre)SQL

- How do we rewrite the name conversion expression into a function?

```
create function full_name(p_fname varchar, p_sname varchar)
returns varchar
as $$
begin
    return case
        when p_fname is null then ''
        else p_fname || ' '
    end || case position('(' in p_sname)
        when 0 then p_sname
        else trim(')' from substr(p_sname, position('(' in p_sname) + 1))
        || ' '
        || trim(substr(p_sname, 1, position('(' in p_sname) - 1))
    end;
end;
$$ language plpgsql;
```

Function name and the parameter list

- Format for variables and parameters: [name] [type]

Function in (Postgre)SQL

- How do we rewrite the name conversion expression into a function?

```
create function full_name(p_fname varchar, p_sname varchar)
returns varchar
as $$
begin
    return case
        when p_fname is null then ''
        else p_fname || ' '
    end || case position('(' in p_sname)
        when 0 then p_sname
        else trim(')' from substr(p_sname, position('(' in p_sname) + 1))
        || ' '
        || trim(substr(p_sname, 1, position('(' in p_sname) - 1))
    end;
end;
$$ language plpgsql;
```

Return type

Function in (Postgre)SQL

- How do we rewrite the name conversion expression into a function?

Body

```
create function full_name(p_fname varchar, p_sname varchar)
returns varchar
as $$
begin
    return case
        when p_fname is null then ''
        else p_fname || ' '
    end || case position('(' in p_sname)
        when 0 then p_sname
        else trim(')' from substr(p_sname, position('(' in p_sname) + 1))
        || ' '
        || trim(substr(p_sname, 1, position('(' in p_sname) - 1))
    end;
end;
$$ language plpgsql;
```

Function in (Postgre)SQL

- How do we rewrite the name conversion expression into a function?

```
create function full_name(p_fname varchar, p_sname varchar)
returns varchar
as $$
begin
    return case
        when p_fname is null then ''
        else p_fname || ' '
    end || case position('(' in p_sname)
        when 0 then p_sname
        else trim(')' from substr(p_sname, position('(' in p_sname) + 1))
        || ' '
        || trim(substr(p_sname, 1, position('(' in p_sname) - 1))
    end;
end;
$$ language plpgsql;
```

A very simple body: return the value of an expression

Function in (Postgre)SQL

- How do we rewrite the name conversion expression into a function?

```
create function full_name(p_fname varchar, p_sname varchar)
returns varchar
as $$
begin
    return case
        when p_fname is null then ''
        else p_fname || ' '
    end || case position('(' in p_sname)
        when 0 then p_sname
        else trim(')' from substr(p_sname, position
        || ' '
        || trim(substr(p_sname, 1, position('(' in
    end;
end;
$$ language plpgsql;
```

A very simple body: return the value of an expression

Procedural extensions provide all the features in a true (procedural) programming languages, such as:

- Variables
- Conditions
- Loops
- Arrays
- Error management
- …

Function in (Postgre)SQL

- How do we rewrite the name conversion expression into a function?

```
create function full_name(p_fname varchar, p_sname varchar)
returns varchar
as $$
begin
    return case
        when p_fname is null then ''
        else p_fname || ' '
    end || case position('(' in p_sname)
        when 0 then p_sname
        else trim(')' from substr(p_sname, position('(' in p_sname) + 1))
        || ' '
        || trim(substr(p_sname, 1, position('(' in p_sname) - 1))
    end;
end;
$$ language plpgsql;
```

Language Type

PostgreSQL supports 4 procedural languages: PL/pgSQL, PL/Tcl, PL/Perl, and PL/Python

- Tcl, Perl, and Python are famous scripting languages in case you don't know

Function in (Postgre)SQL

- How do we rewrite the name conversion expression into a function?

```
create function full_name(p_
returns varchar
as $$
begin
    return case
        when p_fname is null
        else p_fname || ' '
    end || case position('(' in p_sname)
        when 0 then p_sname
        else trim(')' from substr( sname, position('(' in p_sname) + 1))
        || ' '
        || trim(substr(p_sname, 1 osition('(' in p_sname) - 1))
    end;
end;
$$ language plpgsql;
```

```
create function append_test(p_code varchar)
returns varchar
as $$
    if p_code == 'cn':
        return 'China'
    else:
        return 'not China'
$$ language plpython3u;
```

Yes, we can even use Python to write functions

Language Type

PostgreSQL supports 4 procedural languages:
PL/pgSQL, PL/Tcl, PL/Perl, and PL/Python

- Tcl, Perl, and Python are famous scripting languages in case you don’t know

Function in (Postgre)SQL

- Once your function is created, you can use it as if it were any built-in function.

```
select full_name(first_name, surname)
from people
where surname like '%(von)';
```

Function in (Postgre)SQL

- We can run `select` queries in functions
  - Example: design a function “get_country_name” to transform the country codes into country names based on the `countries` table

```
create function get_country_name(p_code varchar)
returns countries.country_name%type
as $$
declare
    v_name countries.country_name%type;
begin
    select country_name
    into v_name
    from countries
    where country_code = p_code;
    return v_name;
end;
$$ language plpgsql;
```

i.e., same type as countries.country_name

```
select get_country_name(country) from movies;
```

Function in (Postgre)SQL

- We can run `select` queries in functions
  - Example: design a function “get_country_name” to transform the country codes into country names based on the `countries` table

```
create function get_country_name(p_code varchar)
returns countries.country_name%type
as $$
declare
    v_name countries.country_name%type;
begin
    select country_name
    into v_name
    from countries
    where country_code = p_code;
    return v_name;
end;
$$ language plpgsql;
```

```
select get_country_name(country) from movies;
```

… seems to be an easy way to get rid of `join` operations?

```
select c.country_name
from countries c join movies m
on c.country_code = m.country;
```

Function in (Postgre)SQL

- A “look-up function” forces a “one row at a time” join which in most cases will be costly

```
select get_country_name(country) from movies;
```

For each row in movies, the `select` query in `get_country_name()` is executed once

More to Read

- We may not cover all the details in functions in the theoretical session, so here are some more materials on procedural programming in PostgreSQL:
  - Lab tutorial on Functions
    - Please read it before your next lab sessions
  - Chapter 5.2 “Functions and Procedures,” Database System Concepts (7th Edition)
  - Chapter 43 “PL/pgSQL,” PostgreSQL Documentation
    - https://www.postgresql.org/docs/current/plpgsql.html

Procedures

Functions vs. Procedures

- Generally,
  - “Function” comes from mathematics
    - … which calculates a value with a given input (or to say, map a value to another)
    - Thus, functions always have a return value
  - ”Procedure” comes from programming
    - … which is used to describe a set of instructions that will be executed in order
    - … and does NOT (necessarily) have a return value
- However,
  - Sometimes, the two terms are interchangeably (used for representing the same thing)
    - e.g., procedures are called functions as well
  - Be careful when seeing both terms
    - Always identify the exact meaning of each term and see whether they have different or the same meaning(s)

Functions and Procedures in (Postgre)SQL

- It follows the general definition of functions and procedures
  - Function: return a value
  - Procedure: return NO value
- However,
  - For some historical reasons, PostgreSQL actually has no implementation specifically for procedures
    - It shares the same mechanism with functions
    - Treats procedures as void functions
  - * But for some other database systems, there are separate implementations for functions and procedures

When to Use Procedures

- For business logics
  - One requirement may need a series of SQL querys and statements
  - Transactions may be used
  - Example: Insert a new movie into the databases
    - `movies` table
      - Basic information for the movie
    - `countries` table
      - Transformation between country names and codes
    - `people` table
      - new actors / directors
    - `credits` table
      - new credit information
    - Problem: Update all the tables? Input validation? Code reuse? Security?

When to Use Procedures

- To add a movie:
  - We may have a series queries to execute when inserting only one movie
  - How about one call for all the processes?
    - Benefit 1: Network overhead
      - When running multiple queries, you are going to waste time chatting over the network with the remote server
    - Benefit 2: Security
      - Prevent users from modifying data otherwise than by calling carefully written and well tested procedures
      - Ensure that users can only modify data via carefully written and well tested procedures

Example: Adding a New Movie

- The information provided for a new movie:
  - Title
  - Year
  - Country Name
    - Note: Name, not code. Country codes are not user-friendly
      - E.g. Which country does “at” represent? ”al”? “ma”? “li”?
  - Director
  - Actor 1
  - Actor 2
    - Let’s assume that only one director and at most two actors are allowed
    - It will be more difficult when the number of people are flexible

A Typical Process

- Insert and check the values, constraints, existence, duplicates, etc
  - A series of inter-related statements

```
select country_code from countries
... -- Look up the country code

insert into movies
... -- Insert a row in the movies table

select peopleid from people
...
insert into credits
... -- Director

select peopleid from people
...
insert into credits
... -- Actor 1

select peopleid from people
...
insert into credits
... -- Actor 2
```

A Typical Process

- How can we pack them into a single execution unit?
  - Minimize communication between client program and database server
    - Client program = DataGrip, `psql`, …

```
select country_code from countries
... -- Look up the country code

insert into movies
... -- Insert a row in the movies table

select peopleid from people
...
insert into credits
... -- Director

select peopleid from people
...
insert into credits
... -- Actor 1

select peopleid from people
...
insert into credits
... -- Actor 2
```

`insert into select`

- One thing to optimize: `insert into ... select`

```
-- when the arities of table 1 and 2 are the same:
insert into table2
select * from table1
where condition;

-- only insert specific columns
insert into table2 (column1, column2, column3, ...)
select column1, column2, column3, ...
from table1
where condition;
```

Optimize the Insertion

- Use `insert into select`

```
insert into movies ...
select country_code, ...
from countries
...

-- insert the director
-- by looking up the people table
insert into credits ...
select peopleid, 'D', ...
from people
...

-- insert the first actor
-- by looking up the people table
insert into credits ...
select peopleid, 'A', ...
from people
...

-- insert the second actor
-- by looking up the people table
insert into credits ...
select peopleid, 'A', ...
from people
...
```

Further Optimize the Insertion

- Use `insert into select`

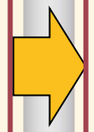
- Combine the queries of `people`

```
insert into movies ...
select country_code, ...
from countries
...

-- insert the director
-- by looking up the people table
insert into credits ...
select peopleid, 'D', ...
from people
...

-- insert the first actor
-- by looking up the people table
insert into credits ...
select peopleid, 'A', ...
from people
...

-- insert the second actor
-- by looking up the people table
insert into credits ...
select peopleid, 'A', ...
from people
...
```



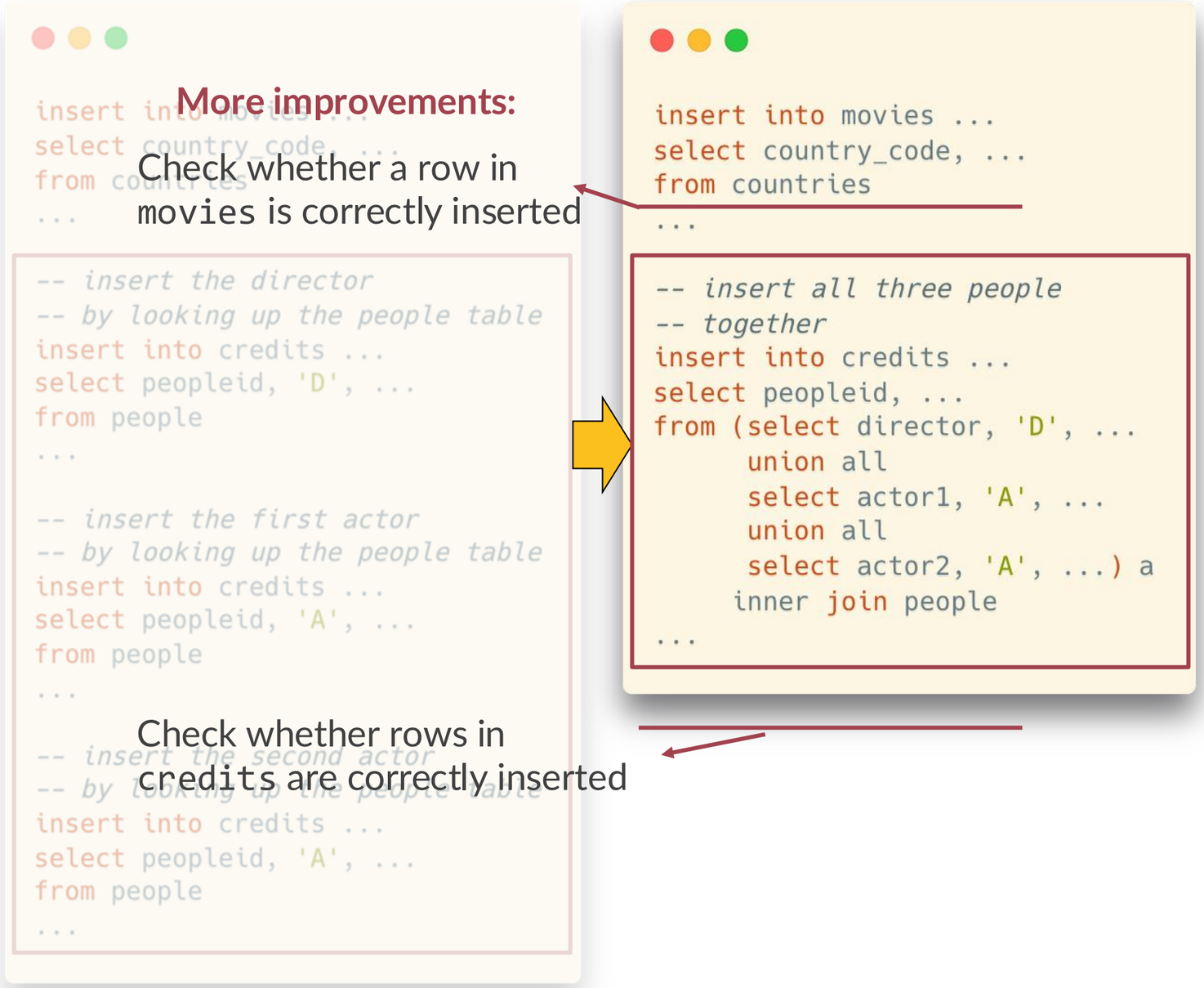
```
insert into movies ...
select country_code, ...
from countries
...

-- insert all three people
-- together
insert into credits ...
select peopleid, ...
from (select director, 'D', ...
      union all
      select actor1, 'A', ...
      union all
      select actor2, 'A', ...) a
     inner join people
...
```

Further Optimize the Insertion

- Use `insert into select`

- Combine the queries of `people`



The Procedure

```
create function movie_registration
           (p_title        varchar,
            p_country_name varchar,
            p_year         int,
            p_director_fn  varchar,
            p_director_sn  varchar,
            p_actor1_fn    varchar,
            p_actor1_sn    varchar,
            p_actor2_fn    varchar,
            p_actor2_sn    varchar)
returns void
as $$
declare
  n_rowcount int;
  n_movieid int;
  n_people int;
begin
    insert into movies(title, country, year_released)
        select p_title, country_code, p_year
        from countries
        where country_name = p_country_name;
    get diagnostics n_rowcount = row_count;

    if n_rowcount = 0
    then
        raise exception 'country not found in table COUNTRIES';
    end if;

    n_movieid := lastval();
    select count(surname)
    into n_people
    from (select p_director_sn as surname
        union all
        select p_actor1_sn as surname
        union all
        select p_actor2_sn as surname) specified_people
    where surname is not null;

    insert into credits(movieid, peopleid, credited_as)
        select n_movieid, people.peopleid, provided.credited_as
        from (select coalesce(p_director_fn, '*') as first_name,
            p_director_sn as surname,
            'D' as credited_as
            union all
            select coalesce(p_actor1_fn, '*') as first_name,
                p_actor1_sn as surname,
                'A' as credited_as
            union all
            select coalesce(p_actor2_fn, '*') as first_name,
                p_actor2_sn as surname,
                'A' as credited_as) provided
            inner join people
        on people.surname = provided.surname
            and coalesce(people.first_name, '*') = provided.first_name
        where provided.surname is not null;

    get diagnostics n_rowcount = row_count;
    if n_rowcount != n_people
    then
        raise exception 'Some people couldn''t be found';
    end if;
end;
$$ language plpgsql;
```

The Procedure

```
create function movie_registration
           (p_title        varchar,
            p_country_name varchar,
            p_year         int,
            p_director_fn  varchar,
            p_director_sn  varchar,
            p_actor1_fn    varchar,
            p_actor1_sn    varchar,
            p_actor2_fn    varchar,
            p_actor2_sn    varchar)
returns void
as $$
declare
  n_rowcount int;
  n_movieid int;
  n_people int;
begin
    insert into movies(title, country, year_released)
        select p_title, country_code, p_year
        from countries
        where country_name = p_country_name;
    get diagnostics n_rowcount = row_count;

    if n_rowcount = 0
    then
        raise exception 'country not found in table COUNTRIES';
    end if;
```

Check whether a row in `movies` is correctly inserted

```
    n_movieid := lastval();
    select count(surname)
    into n_people
    from (select p_director_sn as surname
        union all
        select p_actor1_sn as surname
        union all
        select p_actor2_sn as surname) specified_people
    where surname is not null;

    insert into credits(movieid, peopleid, credited_as)
        select n_movieid, people.peopleid, provided.credited_as
        from (select coalesce(p_director_fn, '*') as first_name,
            p_director_sn as surname,
            'D' as credited_as
            union all
            select coalesce(p_actor1_fn, '*') as first_name,
                p_actor1_sn as surname,
                'A' as credited_as
            union all
            select coalesce(p_actor2_fn, '*') as first_name,
                p_actor2_sn as surname,
                'A' as credited_as) provided
            inner join people
        on people.surname = provided.surname
            and coalesce(people.first_name, '*') = provided.first_name
        where provided.surname is not null;

    get diagnostics n_rowcount = row_count;
    if n_rowcount != n_people
    then
        raise exception 'Some people couldn''t be found';
    end if;
end;
$$ language plpgsql;
```

Check whether rows in `credits` are correctly inserted

Calling Procedures

- In PostgreSQL
  - We can call the procedure interactively by calling it from a SELECT statement (that will return nothing)

```
select movie_registration('The Adventures of Robin Hood',
                          'United States', 1938,
                          'Michael', 'Curtiz',
                          'Errol', 'Flynn',
                          null, null);
```

- We can also call a procedure from another procedure

```
perform movie_registration('The Adventures of Robin Hood',
                           'United States', 1938,
                           'Michael', 'Curtiz',
                           'Errol', 'Flynn',
                           null, null);
```